

EE102 Term Project Report

Youtube Link: <https://youtu.be/L0eVtXn3cmY>

Purpose

The purpose of this project was to design a basic security system. It detects the motion around the object need to be protected and also measure and display the temperature simultaneously.

Methodology

This project is written no other than VHDL. There are two parts in this project: temperature sensor data processing and motion detection. Temperature sensor use is a LM35 which is outputting an analog data so it requires a longer process. And the motion sensor is PIR (Passive Infrared) motion sensor. The tasks that needed to be done as follows:

- 1- Implementing an ADC (Analog to Digital Converter) module
- 2- Seven-segment display for obtained data stemming from temperature sensor
- 3- Conditional statements for detection of the motion

Design Specifications

The main module “xadc.vhd” has 6 inputs and 5 outputs which are as follows:

```
motionsensor : in std_logic;

internalclk : in std_logic;

vauxp6      : in std_logic;

vauxn6      : in std_logic;

vp_in       : in std_logic;

vn_in       : in std_logic;

anode       : out std_logic_vector(3 downto 0);

decimalPoint : out std_logic;

cathode     : out std_logic_vector(6 downto 0);

buzzerout   : out std_logic := '0';

ledout      : out std_logic := '0'
```

There are 2 submodules of main module: “m_b2d.vhd” and “DigitToSeg.vhd”. And “DigitToSeg.vhd” has 5 submodules.

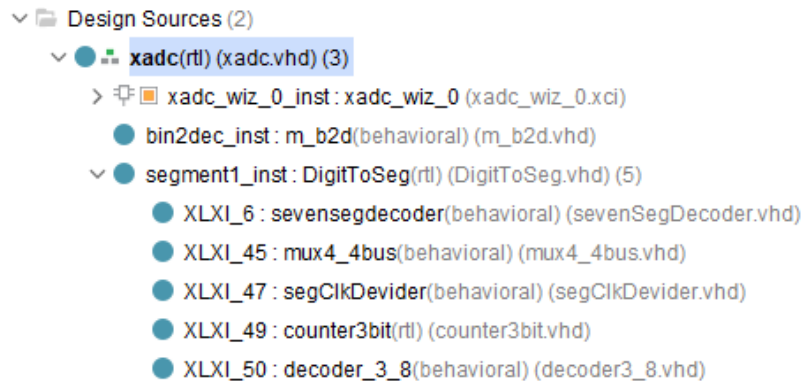


Figure 1: Hierarchy

An ADC Wizard is also created and added under the main module. This file is created by compiler and a read-only module, user is only make selections about ports and other specific process settings. It is created from “IP Catalog” under the “Flow Navigator” part.

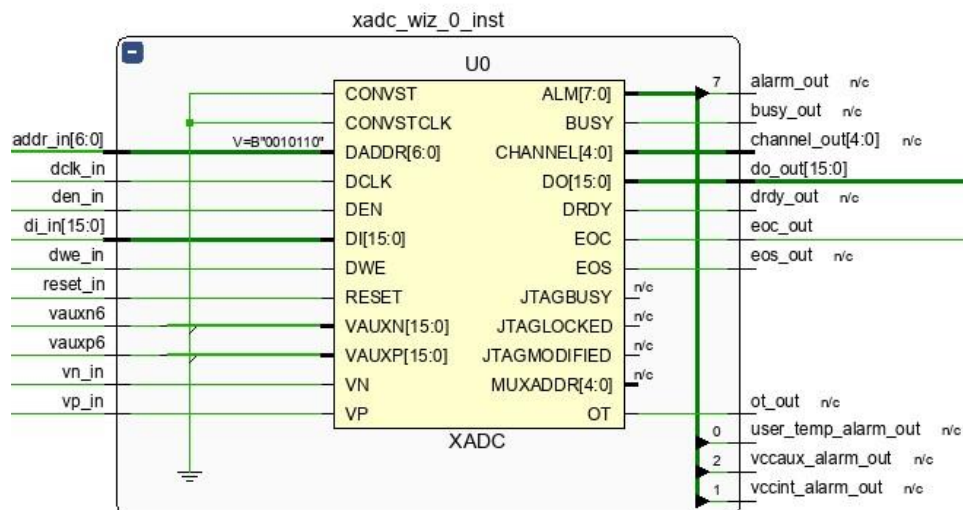


Figure 6: XADC Wizard schematic

The “xadc.vhd” file controls both of the sensors. A sequential expression is written for the motion sensor which makes buzzer and LED –symbolising alarm mechanism- outputs high or low.

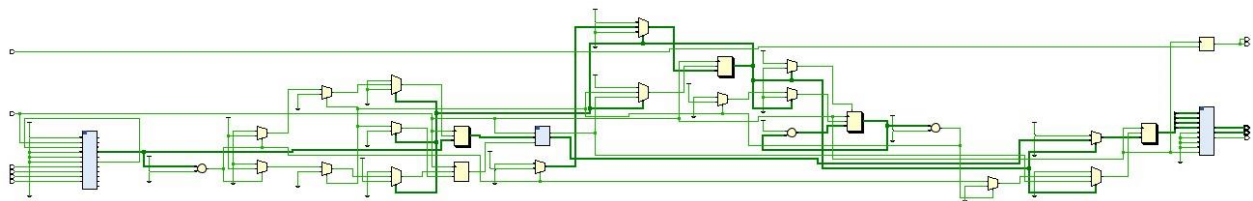


Figure 7: Schematic of the main module

Dual Analog/Digital Pmod (JXADC)

The main purpose of this file is to input analog signal coming from temperature sensor LM35. This sensor outputs 10 mV/°C. This sensor is connected to first positive and negative pins of the ADC ports of BASYS3.

LM35 connection is on highlighted pins named as “vauxp6” (positive) and “vauxn6” (negative).

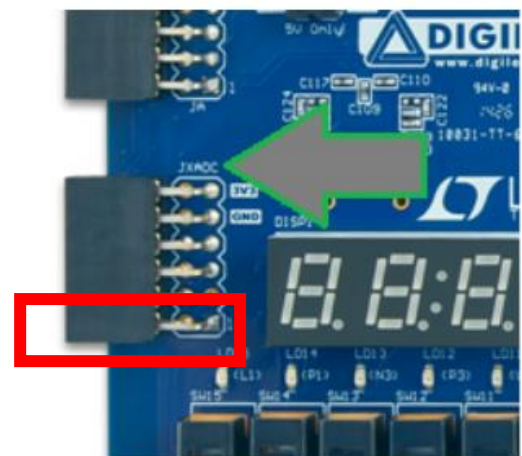


Figure 8: JXADC Analog pins of BASYS3

“xadc.vhd” file inputs this data and converts into digital and sends it to the “m_b2d.vhd” module. This module has 3 inputs (clk, start, din) and 2 outputs (done, dout, dout is declared as buffer) and converts inputted binary number into BCD.

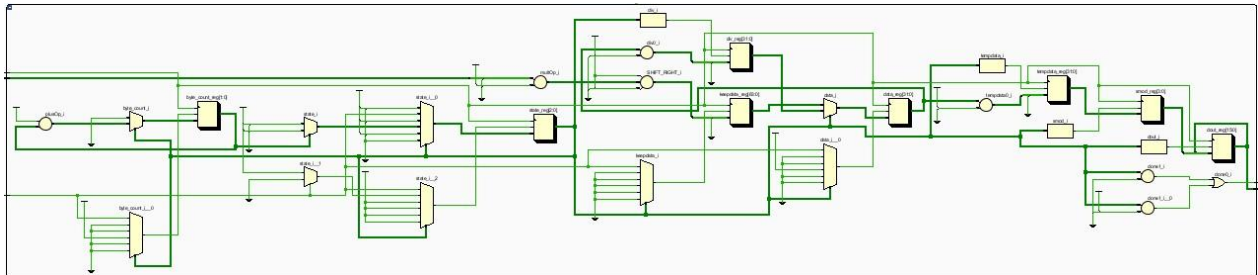


Figure 9: Schematic of binary to BCD converter

The BCD value is sent to “DigitToSeg.vhd” module and it has displays it on the seven-segment display of the BASYS 3 board using 5 submodules.

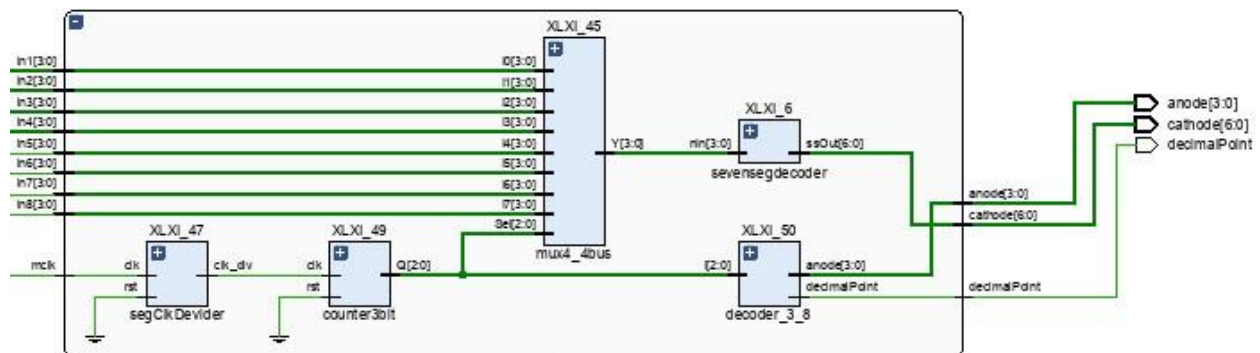


Figure 10: Schematic of Digit-Segment controller module

Next photos are including details of the submodules of “DigitToSeg.vhd” module shown in the previous photo.

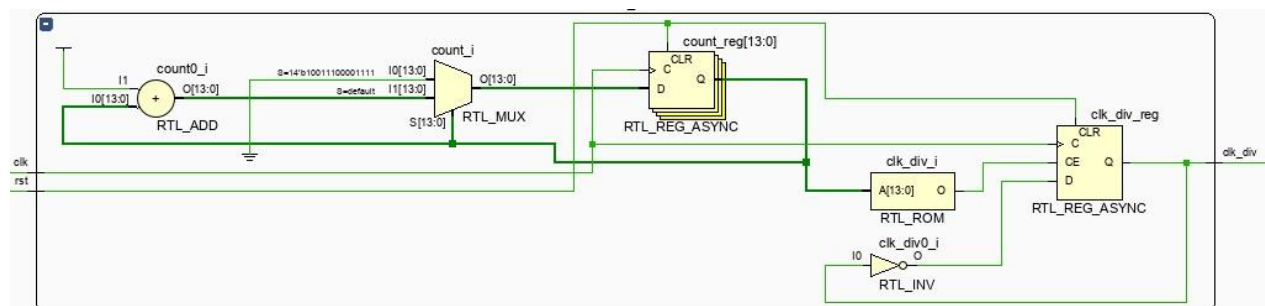


Figure 11: Schematic of segment clock divider module

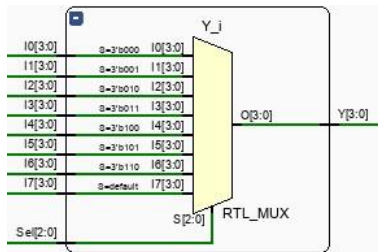


Figure 15: Schematic of multiplexer module

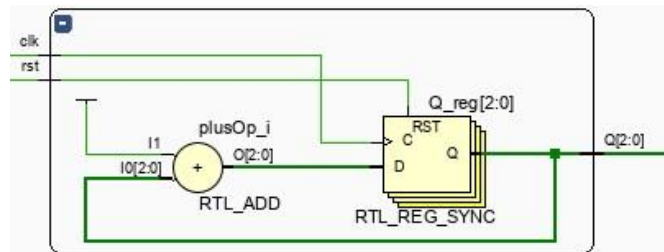


Figure 14: Schematic of counter module

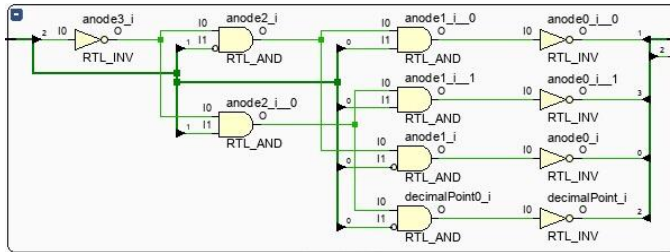


Figure 13: Schematic of decoder module

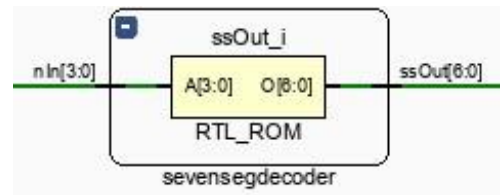


Figure 12: Schematic of seven-segment decoder module

Results

In order to show different temperatures in different cases, 3 things are implemented. In the first case, normal room temperature is measured and displayed. Temperature is about 25.5°C.

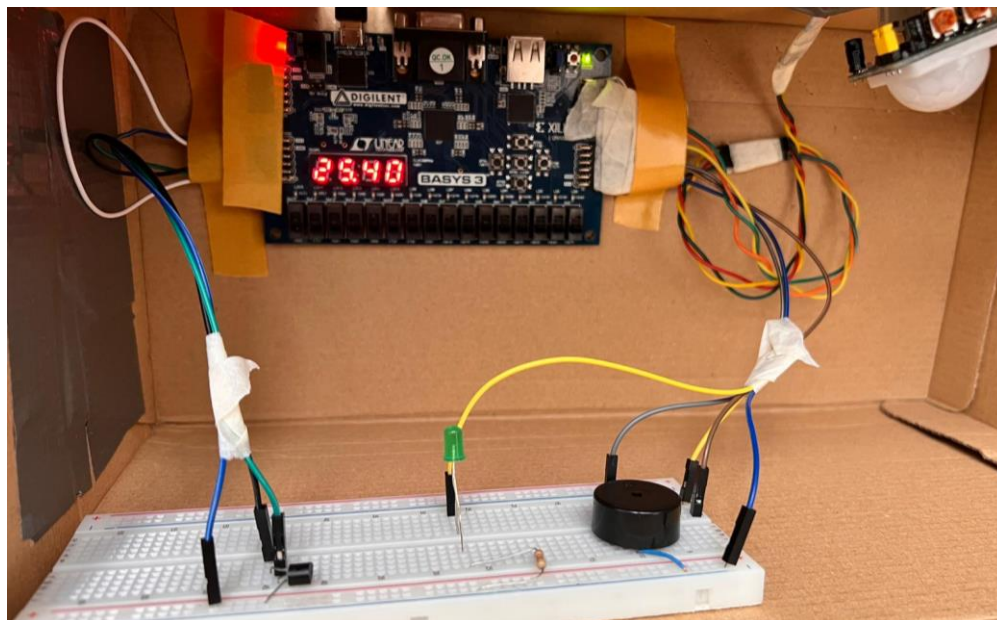


Figure 16: Room temperature is being measured and displayed

In the second case, a small candle is put in the room. Temperature is arisen about 30°C.

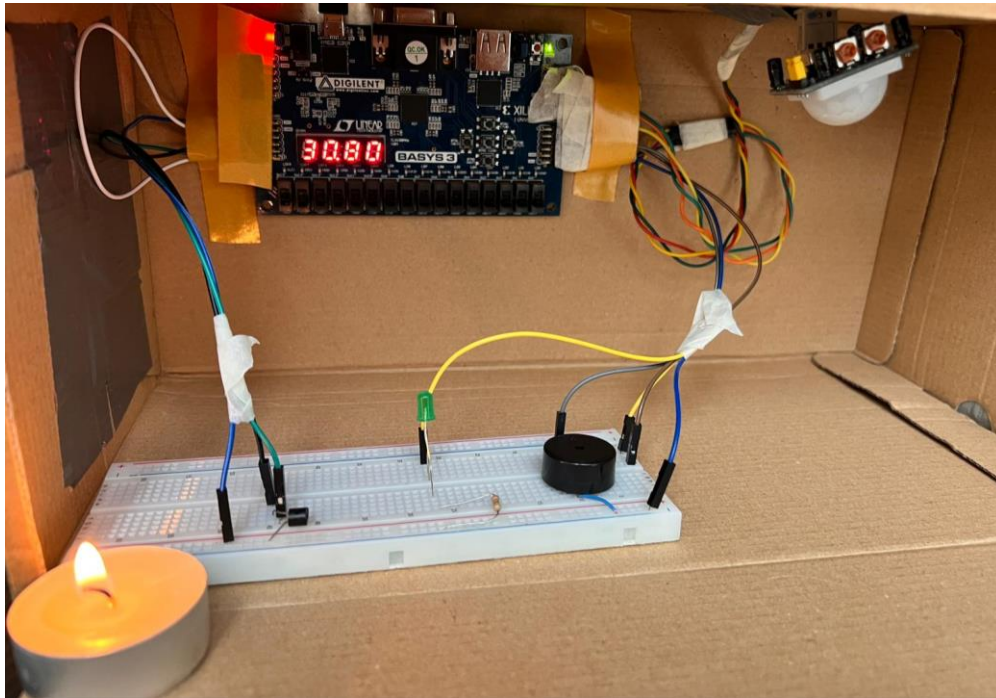


Figure 17: Environment is heated, temperature has arisen

In the third case, a cup of ice cubes are replaced with the candle. It can be observed that temperature fell about 19°C.

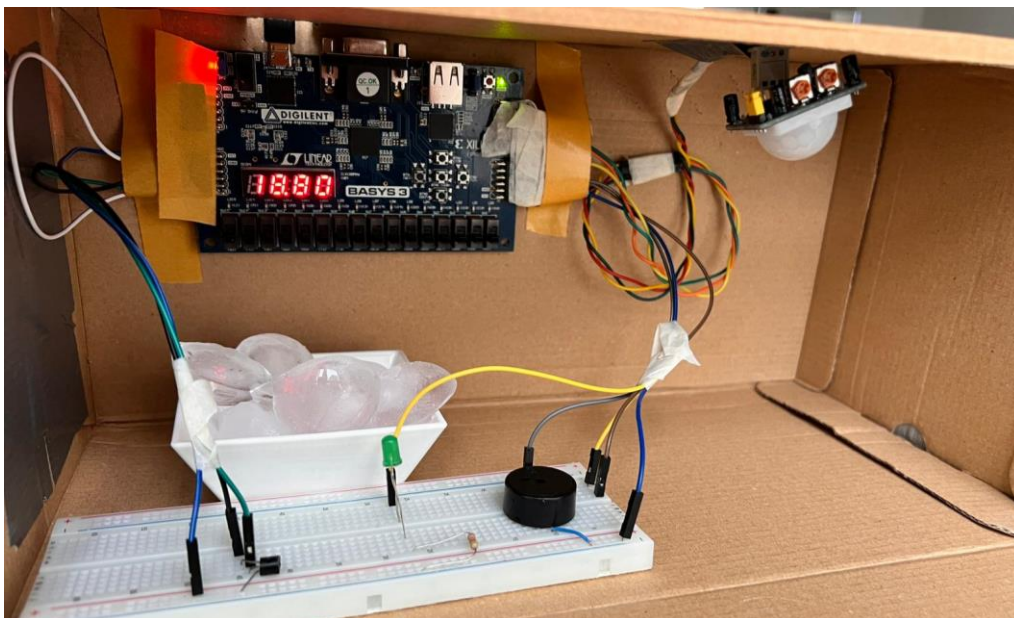


Figure 18: Environment got colder, temperature has fallen

In the second part of the project, motion detection is tested. In the photos below, detected case and undetected case can be seen.

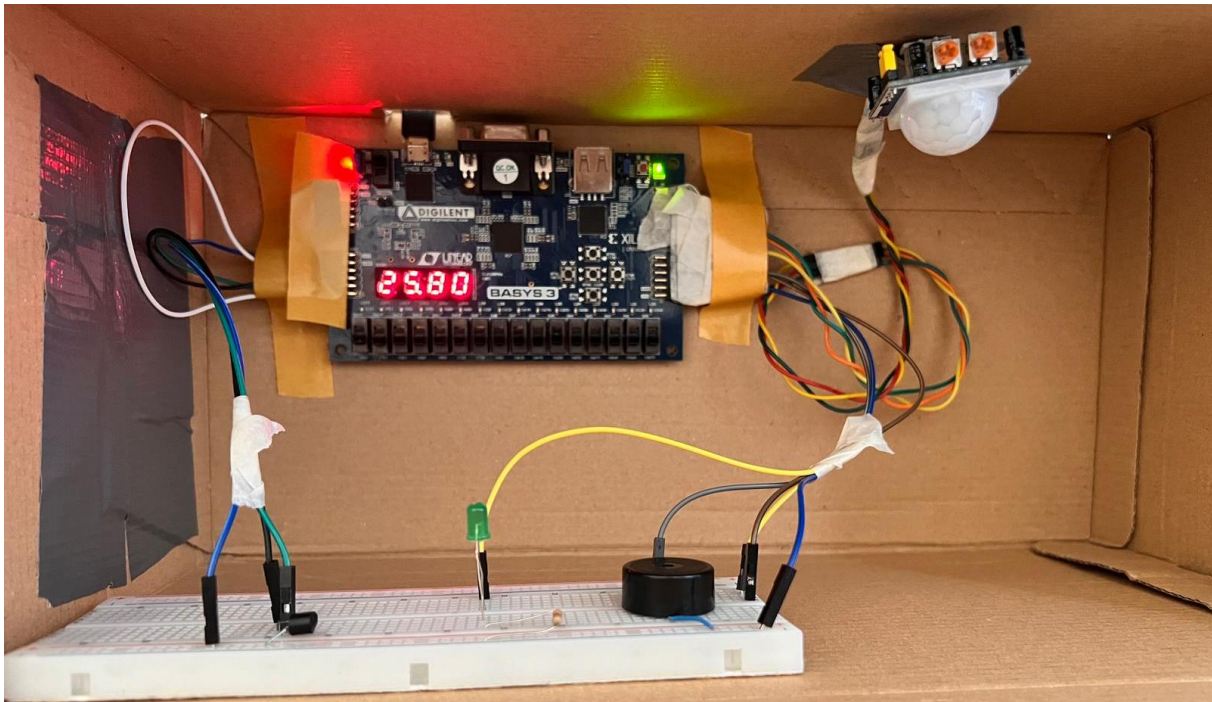


Figure 20: No motion is detected, LED and buzzer symbolizing alarm system is OFF

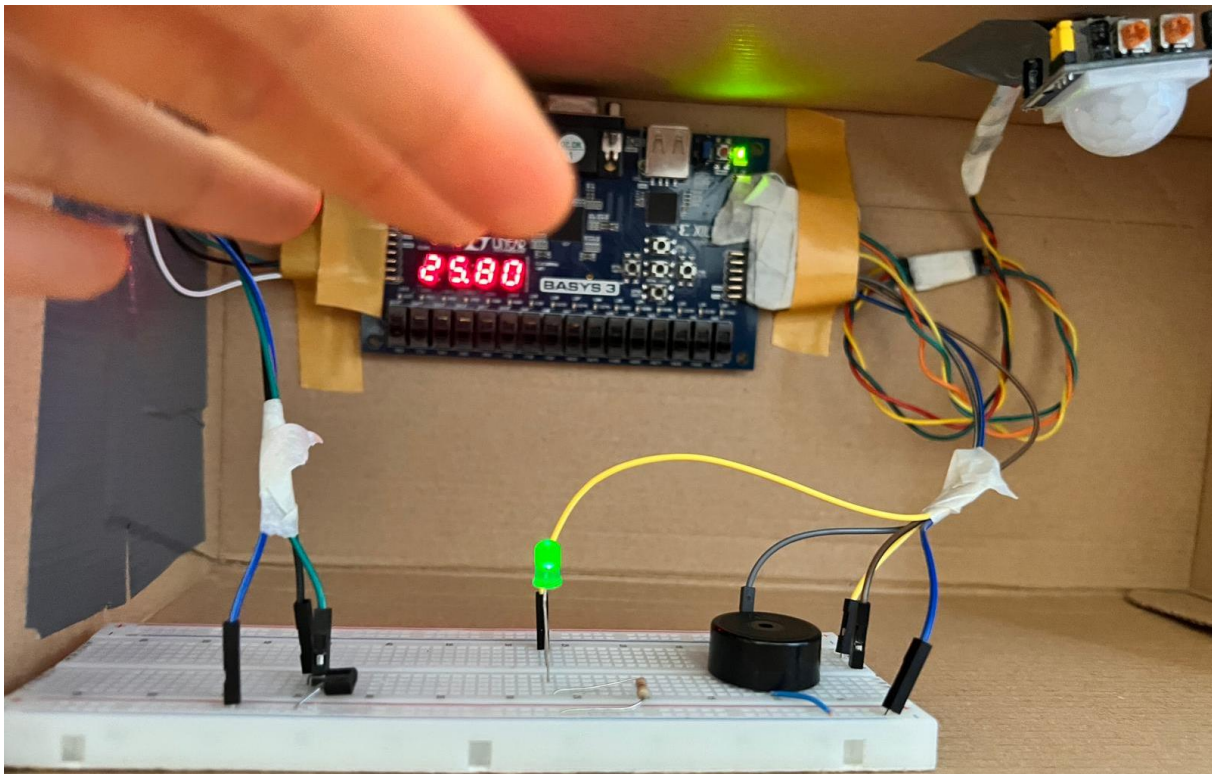


Figure 19: Motion is detected, LED and buzzer symbolizing alarm system is ON

Conclusion

In this project, it was aimed to design a security system keeps valuable items or some specific place safe. The control is made in terms of two risk factors: temperature and motion. To make this design, I inspired by the most famous museums and the pieces exhibited at those places. In theory, my purpose was to design states in which the outputs of the both sensors would be connected to different LED and buzzer combinations and timing would be different to make this project worth to work on and present. I designed the proposal in the beginning as both sensors would give a digital output. However, in reality, I found a temperature sensor, LM35, outputting analog data. ADC conversion seemed more doable than simpler security system since it could be too basic to implement as term project. Because of this reason and instead of finding and buying a new sensor, I used this sensor. Since the output is analog, not digital, I decided not to assign temperature output into a LED and buzzer, but instead, I displayed the environment temperature using seven-segment display feature of the BASYS 3 board. Therefore, instead of the alarm system, ADC conversion became dominant in the project so simple algorithm is written for motion detection instead of complicated, controlled by two control signals (temperature and motion) systems.

During the project, the biggest hardship was analog to digital conversion. It was complicated to implement since I saw and used wizard and IP configurations for the first time. Also the seven segment display was different than what I did in the labs. This time it was not displaying numbers increasing per second, but instead it is displaying what is inputted by the module. Lastly, because of the default configuration of the PIR motion sensor, inconsistencies can be observed. It does not detect the motion following three seconds after it goes low from high, so when a motion happens right after it turns OFF, it is missed. Also it has two calibration potentiometers, one is for detection range (distance) and the other is for staying at high time. These potentiometers were highly sensitive to adjust the wanted value.

More extreme cases could be observed in terms of temperature; however, since the presentation of the project and it is not a professional setup, components are not isolated so I avoided from going for more extreme conditions.

Last of all, as mentioned, this project is a simple security system. It could be implemented with a number of various sensors, buttons, switches etc. in another project.

Appendices

```
library ieee;
```

```
use ieee.std_logic_1164.all;
```

```
use ieee.numeric_std.all;
```

```
entity xadc is
```

```
    port (motionsensor : in std_logic;
```

```
          internalclk  : in std_logic;
```

```
          vauxp6       : in std_logic;
```

```
          vauxn6       : in std_logic;
```

```
          vp_in        : in std_logic;
```

```
          vn_in        : in std_logic;
```

```
          anode        : out std_logic_vector(3 downto 0);
```

```
          decimalPoint : out std_logic;
```

```
          cathode      : out std_logic_vector(6 downto 0);
```

```
          buzzerout    : out std_logic := '0';
```

```
          ledout       : out std_logic := '0'
```

```
    );
```

```
end entity xadc;
```

```
architecture rtl of xadc is
```

```
    signal enable  : std_logic;
```

```
    signal ready   : std_logic;
```

```
    signal data    : std_logic_vector(15 downto 0);
```

```
    signal address: std_logic_vector(6 downto 0) := "0010110";
```

```
-- Seven Segment Display control signals
```

```
type FSMSSD is (idle, frameWait, conversion);
```

```
signal state: FSMSSD:= idle;
```

```
-- constant S_IDLE : std_logic_vector(1 downto 0) := "00";
```

```
-- constant S_FRAME_WAIT : std_logic_vector(1 downto 0) := "01";  
-- constant S_CONVERSION : std_logic_vector(1 downto 0) := "10";
```

```
signal count    : unsigned(32 downto 0);  
signal ssdData : std_logic_vector(15 downto 0);
```

```
-- Binary to decimal converter signals
```

```
signal BCDStart : std_logic;  
signal BCDIn    : std_logic_vector(15 downto 0);  
signal BCDOut   : std_logic_vector(15 downto 0);  
signal BCDDone  : std_logic;
```

```
component xadc_wiz_0 is
```

```
port (  
    di_in : IN STD_LOGIC_VECTOR(15 DOWNTO 0);  
    daddr_in : IN STD_LOGIC_VECTOR(6 DOWNTO 0);  
    den_in : IN STD_LOGIC;  
    dwe_in : IN STD_LOGIC;  
    drdy_out : OUT STD_LOGIC;  
    do_out : OUT STD_LOGIC_VECTOR(15 DOWNTO 0);  
    dclk_in : IN STD_LOGIC;  
    reset_in : IN STD_LOGIC;  
    vp_in : IN STD_LOGIC;  
    vn_in : IN STD_LOGIC;  
    vauxp6 : IN STD_LOGIC;  
    vauxn6 : IN STD_LOGIC;  
    user_temp_alarm_out : OUT STD_LOGIC;  
    vccint_alarm_out : OUT STD_LOGIC;  
    vccaux_alarm_out : OUT STD_LOGIC;  
    ot_out : OUT STD_LOGIC;
```

```
channel_out : OUT STD_LOGIC_VECTOR(4 DOWNTO 0);  
eoc_out : OUT STD_LOGIC;  
alarm_out : OUT STD_LOGIC;  
eos_out : OUT STD_LOGIC;  
busy_out : OUT STD_LOGIC  
);  
end component;
```

component m_b2d is

```
port (  
    clk : in std_logic;  
    start: in std_logic;  
    din : in std_logic_vector(15 downto 0);  
    done : out std_logic;  
    dout : out std_logic_vector(15 downto 0)  
);  
end component;
```

component DigitToSeg is

```
port (  
    in1 : in std_logic_vector(3 downto 0);  
    in2 : in std_logic_vector(3 downto 0);  
    in3 : in std_logic_vector(3 downto 0);  
    in4 : in std_logic_vector(3 downto 0);  
    in5 : in std_logic_vector(3 downto 0);  
    in6 : in std_logic_vector(3 downto 0);  
    in7 : in std_logic_vector(3 downto 0);  
    in8 : in std_logic_vector(3 downto 0);  
    mclk: in std_logic;  
    anode : out std_logic_vector(3 downto 0);
```



```

        decimalPoint : out std_logic;
        cathode : out std_logic_vector(6 downto 0)
    );
end component;

```

```
begin
```

```

xadc_wiz_0_inst: xadc_wiz_0

```

```

    port map (
        di_in => x"0000",
        daddr_in => address,
        den_in => enable,
        dwe_in => '0',
        drdy_out => ready,
        do_out => data,
        dclk_in => internalclk,
        reset_in => '0',
        vp_in => vp_in,
        vn_in => vn_in,
        vauxp6 => vauxp6,
        vauxn6 => vauxn6,
        channel_out => open,
        eoc_out => enable,
        alarm_out => open,
        busy_out => open
    );

```

```

        conversion_process: process (internalclk, count, data, ssdData, BCDStart, BCDIn, BCDDone, BCDOut)
begin

```

```

    if rising_edge(internalclk) then
        case state is

```

```

when idle =>
    state <= frameWait;
    count <= (others => '0');
when frameWait =>
    if (count >= 10000000) then
        if (data > x"FFD0") then
            ssdData <= "0000000000001000";
            state <= idle;
        else
            BCDStart <= '1';
            BCDIn <= data;
            state <= conversion;
        end if;
    else
        count <= count + 1;
    end if;
when others =>
    BCDStart <= '0';
    if BCDDone = '1' then
        ssdData <= BCDDOut;
        state <= idle;
    end if;
end case;
end if;
end process conversion_process;

```

```

bin2dec_inst: m_b2d
port map (
    clk => internalclk,
    start => BCDStart,

```

```

    din  => BCDIn,
    done => BCDDone,
    dout => BCDOut
);

```

segment1_inst: DigitToSeg

```

port map (
    in1    => ssdData(3 downto 0),
    in2    => ssdData(7 downto 4),
    in3    => ssdData(11 downto 8),
    in4    => ssdData(15 downto 12),
    in5    => (others => '0'),
    in6    => (others => '0'),
    in7    => (others => '0'),
    in8    => (others => '0'),
    mclk    => internalclk,
    anode    => anode,
    decimalPoint => decimalPoint,
    cathode    => cathode
);

```

MotionDetection: process(internalclk, motionsensor) begin

```

    if rising_edge(internalclk) then
        if (motionsensor = '1') then
            buzzerout <= '1';
            ledout  <= '1';
        else
            buzzerout <= '0';
            ledout <= '0';
        end if;
    end if;
end if;

```

```
end process;  
end architecture rtl;
```

Code 1: Main Module

```
-- file: xadc_wiz_0.vhd  
-- (c) Copyright 2009 - 2013 Xilinx, Inc. All rights reserved.  
--  
-- This file contains confidential and proprietary information  
-- of Xilinx, Inc. and is protected under U.S. and  
-- international copyright and other intellectual property  
-- laws.  
--  
-- DISCLAIMER  
-- This disclaimer is not a license and does not grant any  
-- rights to the materials distributed herewith. Except as  
-- otherwise provided in a valid license issued to you by  
-- Xilinx, and to the maximum extent permitted by applicable  
-- law: (1) THESE MATERIALS ARE MADE AVAILABLE "AS IS" AND  
-- WITH ALL FAULTS, AND XILINX HEREBY DISCLAIMS ALL WARRANTIES  
-- AND CONDITIONS, EXPRESS, IMPLIED, OR STATUTORY, INCLUDING  
-- BUT NOT LIMITED TO WARRANTIES OF MERCHANTABILITY, NON-  
-- INFRINGEMENT, OR FITNESS FOR ANY PARTICULAR PURPOSE; and  
-- (2) Xilinx shall not be liable (whether in contract or tort,  
-- including negligence, or under any other theory of  
-- liability) for any loss or damage of any kind or nature  
-- related to, arising under or in connection with these  
-- materials, including for any direct, or any indirect,  
-- special, incidental, or consequential loss or damage  
-- (including loss of data, profits, goodwill, or any type of  
-- loss or damage suffered as a result of any action brought
```

-- by a third party) even if such damage or loss was
-- reasonably foreseeable or Xilinx had been advised of the
-- possibility of the same.

--

-- CRITICAL APPLICATIONS

-- Xilinx products are not designed or intended to be fail-
-- safe, or for use in any application requiring fail-safe
-- performance, such as life-support or safety devices or
-- systems, Class III medical devices, nuclear facilities,
-- applications related to the deployment of airbags, or any
-- other applications that could lead to death, personal
-- injury, or severe property or environmental damage
-- (individually and collectively, "Critical
-- Applications"). Customer assumes the sole risk and
-- liability of any use of Xilinx products in Critical
-- Applications, subject only to applicable laws and
-- regulations governing limitations on product liability.

--

-- THIS COPYRIGHT NOTICE AND DISCLAIMER MUST BE RETAINED AS
-- PART OF THIS FILE AT ALL TIMES.

library ieee;

use ieee.std_logic_1164.all;

use ieee.numeric_std.all;

Library UNISIM;

use UNISIM.VCOMPONENTS.ALL;

entity xadc_wiz_0 is

port

(

```

    daddr_in      : in STD_LOGIC_VECTOR (6 downto 0); -- Address bus for the dynamic reconfiguration
port
    den_in       : in STD_LOGIC;                      -- Enable Signal for the dynamic reconfiguration port
    di_in        : in STD_LOGIC_VECTOR (15 downto 0); -- Input data bus for the dynamic reconfiguration
port
    dwe_in       : in STD_LOGIC;                      -- Write Enable for the dynamic reconfiguration port
    do_out       : out STD_LOGIC_VECTOR (15 downto 0); -- Output data bus for dynamic reconfiguration
port
    drdy_out     : out STD_LOGIC;                    -- Data ready signal for the dynamic reconfiguration port
    dclk_in      : in STD_LOGIC;                    -- Clock input for the dynamic reconfiguration port
    reset_in     : in STD_LOGIC;                    -- Reset signal for the System Monitor control logic
    vauxp6       : in STD_LOGIC;                    -- Auxiliary Channel 6
    vauxn6       : in STD_LOGIC;
    busy_out     : out STD_LOGIC;                    -- ADC Busy signal
    channel_out  : out STD_LOGIC_VECTOR (4 downto 0); -- Channel Selection Outputs
    eoc_out      : out STD_LOGIC;                    -- End of Conversion Signal
    eos_out      : out STD_LOGIC;                    -- End of Sequence Signal
    ot_out       : out STD_LOGIC;                    -- Over-Temperature alarm output
    vccaux_alarm_out : out STD_LOGIC;                -- VCCAUX-sensor alarm output
    vccint_alarm_out : out STD_LOGIC;                -- VCCINT-sensor alarm output
    user_temp_alarm_out : out STD_LOGIC;              -- Temperature-sensor alarm output
    alarm_out    : out STD_LOGIC;                    -- OR'ed output of all the Alarms
    vp_in       : in STD_LOGIC;                    -- Dedicated Analog Input Pair
    vn_in       : in STD_LOGIC
);
end xadc_wiz_0;

```

architecture xilinx of xadc_wiz_0 is

```

attribute CORE_GENERATION_INFO : string;

attribute CORE_GENERATION_INFO of xilinx : architecture is
"xadc_wiz_0,xadc_wiz_v3_3_8,{component_name=xadc_wiz_0,enable_axi=false,enable_axi4stream=fals

```



```
e,dclk_frequency=100,enable_busy=true,enable_convst=false,enable_convstclk=false,enable_dclk=true,
enable_drp=true,enable_eoc=true,enable_eos=true,enable_vbram_alararm=false,enable_vccddro_alararm=false,enable_Vccint_Alaram=true,enable_Vccaux_alararm=trueenable_vccpaux_alararm=false,enable_vccpint_alararm=false,ot_alararm=true,user_temp_alararm=true,timing_mode=continuous,channel_averaging=None,sequencer_mode=off,startup_channel_selection=single_channel}";
```

```
signal FLOAT_VBRAM_ALARM : std_logic;
signal FLOAT_MUXADDR : std_logic_vector (4 downto 0);
signal aux_channel_p : std_logic_vector (15 downto 0);
signal aux_channel_n : std_logic_vector (15 downto 0);
signal alm_int : std_logic_vector (7 downto 0);
```

```
begin
```

```
alarm_out <= alm_int(7);
vccaux_alarm_out <= alm_int(2);
vccint_alarm_out <= alm_int(1);
user_temp_alarm_out <= alm_int(0);
```

```
aux_channel_p(0) <= '0';
aux_channel_n(0) <= '0';
```

```
aux_channel_p(1) <= '0';
aux_channel_n(1) <= '0';
```

```
aux_channel_p(2) <= '0';
aux_channel_n(2) <= '0';
```

```
aux_channel_p(3) <= '0';
aux_channel_n(3) <= '0';
```

```
aux_channel_p(4) <= '0';  
aux_channel_n(4) <= '0';
```

```
aux_channel_p(5) <= '0';  
aux_channel_n(5) <= '0';
```

```
aux_channel_p(6) <= vauxp6;  
aux_channel_n(6) <= vauxn6;
```

```
aux_channel_p(7) <= '0';  
aux_channel_n(7) <= '0';
```

```
aux_channel_p(8) <= '0';  
aux_channel_n(8) <= '0';
```

```
aux_channel_p(9) <= '0';  
aux_channel_n(9) <= '0';
```

```
aux_channel_p(10) <= '0';  
aux_channel_n(10) <= '0';
```

```
aux_channel_p(11) <= '0';  
aux_channel_n(11) <= '0';
```

```
aux_channel_p(12) <= '0';  
aux_channel_n(12) <= '0';
```

```
aux_channel_p(13) <= '0';  
aux_channel_n(13) <= '0';
```

```
aux_channel_p(14) <= '0';  
aux_channel_n(14) <= '0';
```

```
aux_channel_p(15) <= '0';  
aux_channel_n(15) <= '0';
```

U0 : XADC

generic map(

```
  INIT_40 => X"0016", -- config reg 0  
  INIT_41 => X"3100", -- config reg 1  
  INIT_42 => X"0400", -- config reg 2  
  INIT_48 => X"0100", -- Sequencer channel selection  
  INIT_49 => X"0000", -- Sequencer channel selection  
  INIT_4A => X"0000", -- Sequencer Average selection  
  INIT_4B => X"0000", -- Sequencer Average selection  
  INIT_4C => X"0000", -- Sequencer Bipolar selection  
  INIT_4D => X"0000", -- Sequencer Bipolar selection  
  INIT_4E => X"0000", -- Sequencer Acq time selection  
  INIT_4F => X"0000", -- Sequencer Acq time selection  
  INIT_50 => X"B5ED", -- Temp alarm trigger  
  INIT_51 => X"57E4", -- Vccint upper alarm limit  
  INIT_52 => X"A147", -- Vccaux upper alarm limit  
  INIT_53 => X"CA33", -- Temp alarm OT upper  
  INIT_54 => X"A93A", -- Temp alarm reset  
  INIT_55 => X"52C6", -- Vccint lower alarm limit  
  INIT_56 => X"9555", -- Vccaux lower alarm limit  
  INIT_57 => X"AE4E", -- Temp alarm OT reset  
  INIT_58 => X"5999", -- Vccb ram upper alarm limit  
  INIT_5C => X"5111", -- Vccb ram lower alarm limit  
  SIM_DEVICE => "7SERIES",
```

```
SIM_MONITOR_FILE => "design.txt"
```

```
)
```

```
port map (
```

```
    CONVST      => '0',
```

```
    CONVSTCLK    => '0',
```

```
    DADDR(6 downto 0) => daddr_in(6 downto 0),
```

```
    DCLK         => dclk_in,
```

```
    DEN          => den_in,
```

```
    DI(15 downto 0)  => di_in(15 downto 0),
```

```
    DWE          => dwe_in,
```

```
    RESET        => reset_in,
```

```
    VAUXN(15 downto 0) => aux_channel_n(15 downto 0),
```

```
    VAUXP(15 downto 0) => aux_channel_p(15 downto 0),
```

```
    ALM          => alm_int,
```

```
    BUSY         => busy_out,
```

```
    CHANNEL(4 downto 0) => channel_out(4 downto 0),
```

```
    DO(15 downto 0)  => do_out(15 downto 0),
```

```
    DRDY         => drdy_out,
```

```
    EOC          => eoc_out,
```

```
    EOS          => eos_out,
```

```
    JTAGBUSY     => open,
```

```
    JTAGLOCKED   => open,
```

```
    JTAGMODIFIED => open,
```

```
    OT           => ot_out,
```

```
    MUXADDR      => FLOAT_MUXADDR,
```

```
    VN           => vn_in,
```

```
    VP           => vp_in
```

```
);
```

```
end xilinx;
```

Code 2: Wizard module

```
library ieee;
```

```
use ieee.std_logic_1164.all;
```

```
use ieee.numeric_std.all;
```

```
entity m_b2d is
```

```
  port (
```

```
    clk : in std_logic;
```

```
    start : in std_logic;
```

```
    din : in std_logic_vector(15 downto 0);
```

```
    done : out std_logic;
```

```
    dout : buffer std_logic_vector(15 downto 0) register
```

```
  );
```

```
end entity m_b2d;
```

```
architecture behavioral of m_b2d is
```

```
  type FSM is (sidle, sdone, sdivide, snextDigit, sconvert);
```

```
  signal state : FSM := sidle;
```

```
  signal data : unsigned(31 downto 0);
```

```
  signal div : unsigned(31 downto 0);
```

```
  signal smod : unsigned(3 downto 0);
```

```
  signal byte_count : unsigned(1 downto 0);
```

```
  signal oodin : unsigned(17 downto 0) ;
```

```
  signal tempdata: unsigned(31 downto 0);
```

```
  signal keepdata: unsigned(63 downto 0);
```

```
begin
```

```
  done <= '1' when (state = sidle or state = sdone) else '0';
```

```
  oodin <= unsigned("00" & din);
```

```

process(clk) begin
  if rising_edge(clk) then
    case state is
      when sidle =>
        if start = '1' then
          state <= sdivide;

          keepdata <= shift_right(("0000000000000000" & unsigned(din))*1000, 16) ;

          data <= keepdata(31 downto 0);

          byte_count <= "00";

        end if;
      when sdone =>
        if start = '0' then
          state <= sidle;

        end if;
      when sdivide =>
        div <= data / 10;

        tempdata <= data mod 10 ;

        smod <= tempdata(3 downto 0);

        state <= sconvert;
      when snextDigit =>
        if byte_count = "11" then
          state <= sdone;

        else
          state <= sdivide;

        end if;

        data <= div;

        byte_count <= byte_count + 1;
      when sconvert =>
        dout(11 downto 0) <= dout(15 downto 4);
    end case;
  end if;
end process;

```

```

        dout(15 downto 12) <= std_logic_vector(smod(3 downto 0));

        state <= snextDigit;

    when others =>
        state <= sidle;

    end case;

end if;

end process;

end architecture behavioral;

```

Code 3: Binary to BCD convertor module

```

library ieee;

use ieee.std_logic_1164.all;

use ieee.numeric_std.all;

entity DigitToSeg is

    port (

        in1, in2, in3, in4, in5, in6, in7, in8: in std_logic_vector(3 downto 0);

        mclk: in std_logic;

        anode: out std_logic_vector(3 downto 0);

        cathode: out std_logic_vector(6 downto 0);

        decimalPoint: out std_logic

    );

end entity DigitToSeg;

```

architecture rtl of DigitToSeg is

component sevensegdecoder is

```

    port (

        nIn: in std_logic_vector(3 downto 0);

        ssOut: out std_logic_vector(6 downto 0)

    );

```

```
end component sevensegdecoder;
```

```
component mux4_4bus is
```

```
  port (  
    I0, I1, I2, I3, I4, I5, I6, I7: in std_logic_vector(3 downto 0);  
    Sel: in std_logic_vector(2 downto 0);  
    Y: out std_logic_vector(3 downto 0)  
  );
```

```
end component mux4_4bus;
```

```
component segClkDevider is
```

```
  port (  
    clk, rst: in std_logic;  
    clk_div: out std_logic  
  );
```

```
end component segClkDevider;
```

```
component counter3bit is
```

```
  port (  
    clk, rst: in std_logic;  
    Q: out std_logic_vector(2 downto 0)  
  );
```

```
end component counter3bit;
```

```
component decoder_3_8 is
```

```
  port (  
    I: in std_logic_vector(2 downto 0);  
    decimalPoint: out std_logic;  
    anode: out std_logic_vector(3 downto 0)
```



```

    );
end component decoder_3_8;

signal dividedclk: std_logic;

signal muxout: std_logic_vector(3 downto 0);

signal counterout: std_logic_vector(2 downto 0);

begin

    XLXI_6: sevensegdecoder port map (
        nIn => muxout(3 downto 0),
        ssOut => cathode(6 downto 0));

    XLXI_45: mux4_4bus port map (
        I0 => in1,
        I1 => in2,
        I2 => in3,
        I3 => in4,
        I4 => in5,
        I5 => in6,
        I6 => in7,
        I7 => in8,
        Sel => counterout(2 downto 0),
        Y => muxout(3 downto 0));

    XLXI_47: segClkDevider port map (
        clk => mclk,
        rst => '0', -- Replace with proper reset signal if available
        clk_div => dividedclk);

```

```

XLXI_49: counter3bit port map (
    clk => dividedclk,
    rst => '0', -- Replace with proper reset signal if available
    Q => counterout(2 downto 0));

XLXI_50: decoder_3_8 port map (
    I => counterout(2 downto 0),
    decimalPoint => decimalPoint,
    anode => anode(3 downto 0));
end architecture rtl;

```

Code 4: Segment&digit controller module

```

library ieee;
use ieee.std_logic_1164.all;

entity sevensegdecoder is
    port (
        nIn : in std_logic_vector(3 downto 0);
        ssOut : out std_logic_vector(6 downto 0)
    );
end entity sevensegdecoder;

architecture behavioral of sevensegdecoder is
begin
    process(nIn)
    begin
        case nIn is
            when "0000" => ssOut <= "1000000";

```

```

    when "0001" => ssOut <= "1111001";
    when "0010" => ssOut <= "0100100";
    when "0011" => ssOut <= "0110000";
    when "0100" => ssOut <= "0011001";
    when "0101" => ssOut <= "0010010";
    when "0110" => ssOut <= "0000010";
    when "0111" => ssOut <= "1111000";
    when "1000" => ssOut <= "0000000";
    when "1001" => ssOut <= "0011000";
    when "1010" => ssOut <= "0001000";
    when "1011" => ssOut <= "0000011";
    when "1100" => ssOut <= "1000110";
    when "1101" => ssOut <= "0100001";
    when "1110" => ssOut <= "0000110";
    when "1111" => ssOut <= "0001110";
    when others => ssOut <= "1001001";

end case;

end process;

end architecture behavioral;

```

Code 5: Seven-segment decoder module

```

library ieee;
use ieee.std_logic_1164.all;

entity mux4_4bus is
    port (
        I0, I1, I2, I3, I4, I5, I6, I7 : in std_logic_vector(3 downto 0);
        Sel : in std_logic_vector(2 downto 0);
        Y : out std_logic_vector(3 downto 0)
    );

```

```

end entity mux4_4bus;

architecture behavioral of mux4_4bus is
begin
    process (I0, I1, I2, I3, I4, I5, I6, I7, Sel)
    begin
        case Sel is
            when "000" => Y <= I0;
            when "001" => Y <= I1;
            when "010" => Y <= I2;
            when "011" => Y <= I3;
            when "100" => Y <= I4;
            when "101" => Y <= I5;
            when "110" => Y <= I6;
            when others => Y <= I7;
        end case;
    end process;
end architecture behavioral;

```

Code 6: MUX module

```

library ieee;
use ieee.std_logic_1164.all;

entity segClkDeviver is
    port (
        clk    : in std_logic;
        rst    : in std_logic;
        clk_div : buffer std_logic
    );
end entity segClkDeviver;

```

architecture behavioral of segClkDeviver is

```
    constant constantNumber : integer := 10000;
```

```
    signal count : integer range 0 to constantNumber - 1 := 0;
```

```
begin
```

```
    process (clk, rst)
```

```
    begin
```

```
        if rst = '1' then
```

```
            count <= 0;
```

```
        elsif rising_edge(clk) then
```

```
            if count = constantNumber - 1 then
```

```
                count <= 0;
```

```
            else
```

```
                count <= count + 1;
```

```
            end if;
```

```
        end if;
```

```
    end process;
```

```
    process (clk, rst)
```

```
    begin
```

```
        if rst = '1' then
```

```
            clk_div <= '0';
```

```
        elsif rising_edge(clk) then
```

```
            if count = constantNumber - 1 then
```

```
                clk_div <= not clk_div;
```

```
            else
```

```
                clk_div <= clk_div;
```

```
            end if;
```

```
        end if;
```

```
    end process;  
end architecture behavioral;
```

Code 7: Clock divider module

```
library ieee;  
use ieee.std_logic_1164.all;  
use ieee.numeric_std.all;  
  
entity counter3bit is  
    port (  
        clk, rst: in std_logic;  
        Q: buffer std_logic_vector(2 downto 0)  
    );  
end entity counter3bit;
```

```
architecture rtl of counter3bit is  
begin  
  
    process (clk)  
    begin  
        if rising_edge(clk) then  
            if rst = '1' then  
                Q <= "000";  
            else  
                Q <= std_logic_vector(unsigned(Q) + 1);  
            end if;  
        end if;  
    end process;  
end architecture rtl;
```

Code 8: Counter module

```

library ieee;

use ieee.std_logic_1164.all;

entity decoder_3_8 is
    port (
        I : in std_logic_vector(2 downto 0);
        anode : out std_logic_vector(3 downto 0);
        decimalPoint : out std_logic
    );
end entity decoder_3_8;

architecture behavioral of decoder_3_8 is
begin
    anode(0) <= not (not I(2) and not I(1) and not I(0));
    anode(1) <= not (not I(2) and not I(1) and I(0));
    anode(2) <= not (not I(2) and I(1) and not I(0));
    anode(3) <= not (not I(2) and I(1) and I(0));

    decimalPoint <= not (not I(2) and I(1) and not I(0));
end architecture behavioral;

```

Code 9: SSD controller module

```

# Clock signal

set_property PACKAGE_PIN W5 [get_ports internalclk]
set_property IOSTANDARD LVCMOS33 [get_ports internalclk]
create_clock -add -name sys_clk_pin -period 10.00 -waveform {0 5} [get_ports internalclk]

#7 segment display

set_property PACKAGE_PIN W7 [get_ports {cathode[0]}]

```

```
        set_property IOSTANDARD LVCMOS33 [get_ports {cathode[0]}]
set_property PACKAGE_PIN W6 [get_ports {cathode[1]}]
        set_property IOSTANDARD LVCMOS33 [get_ports {cathode[1]}]
set_property PACKAGE_PIN U8 [get_ports {cathode[2]}]
        set_property IOSTANDARD LVCMOS33 [get_ports {cathode[2]}]
set_property PACKAGE_PIN V8 [get_ports {cathode[3]}]
        set_property IOSTANDARD LVCMOS33 [get_ports {cathode[3]}]
set_property PACKAGE_PIN U5 [get_ports {cathode[4]}]
        set_property IOSTANDARD LVCMOS33 [get_ports {cathode[4]}]
set_property PACKAGE_PIN V5 [get_ports {cathode[5]}]
        set_property IOSTANDARD LVCMOS33 [get_ports {cathode[5]}]
set_property PACKAGE_PIN U7 [get_ports {cathode[6]}]
        set_property IOSTANDARD LVCMOS33 [get_ports {cathode[6]}]
```

```
set_property PACKAGE_PIN V7 [get_ports decimalPoint]
        set_property IOSTANDARD LVCMOS33 [get_ports decimalPoint]
```

```
set_property PACKAGE_PIN U2 [get_ports {anode[0]}]
        set_property IOSTANDARD LVCMOS33 [get_ports {anode[0]}]
set_property PACKAGE_PIN U4 [get_ports {anode[1]}]
        set_property IOSTANDARD LVCMOS33 [get_ports {anode[1]}]
set_property PACKAGE_PIN V4 [get_ports {anode[2]}]
        set_property IOSTANDARD LVCMOS33 [get_ports {anode[2]}]
set_property PACKAGE_PIN W4 [get_ports {anode[3]}]
        set_property IOSTANDARD LVCMOS33 [get_ports {anode[3]}]
```

```
##Pmod Header JC
```

```
##Sch name = JC1
```

```
set_property PACKAGE_PIN K17 [get_ports {motionsensor}]
```



```
        set_property IOSTANDARD LVCMOS33 [get_ports {motionsensor}]
#Sch name = JC2
set_property PACKAGE_PIN M18 [get_ports {ledout}]
        set_property IOSTANDARD LVCMOS33 [get_ports {ledout}]
#Sch name = JC3
set_property PACKAGE_PIN N17 [get_ports {buzzerout}]
        set_property IOSTANDARD LVCMOS33 [get_ports {buzzerout}]

#Pmod Header JXADC
#Sch name = XA1_P
set_property PACKAGE_PIN J3 [get_ports {vauxp6}]
        set_property IOSTANDARD LVCMOS33 [get_ports {vauxp6}]
#Sch name = XA2_P
#set_property PACKAGE_PIN L3 [get_ports {vauxp14}]
#        set_property IOSTANDARD LVCMOS33 [get_ports {vauxp14}]
##Sch name = XA3_P
#set_property PACKAGE_PIN M2 [get_ports {vauxp7}]
#        set_property IOSTANDARD LVCMOS33 [get_ports {vauxp7}]
##Sch name = XA4_P
#set_property PACKAGE_PIN N2 [get_ports {vauxp15}]
#        set_property IOSTANDARD LVCMOS33 [get_ports {vauxp15}]
#Sch name = XA1_N
set_property PACKAGE_PIN K3 [get_ports {vauxn6}]
        set_property IOSTANDARD LVCMOS33 [get_ports {vauxn6}]
#Sch name = XA2_N
#set_property PACKAGE_PIN M3 [get_ports {vauxn14}]
#        set_property IOSTANDARD LVCMOS33 [get_ports {vauxn14}]
##Sch name = XA3_N
#set_property PACKAGE_PIN M1 [get_ports {vauxn7}]
```

```
#      set_property IOSTANDARD LVCMOS33 [get_ports {vauxn7}]  
  
##Sch name = XA4_N  
  
#set_property PACKAGE_PIN N1 [get_ports {vauxn15}]  
  
#      set_property IOSTANDARD LVCMOS33 [get_ports {vauxn15}]
```

Code 10: Constraint File

References

“Basys 3 Reference Manual.” Digilent. Accessed on June 3, 2023.

<https://digilent.com/reference/programmable-logic/basys-3/reference-manual>

“Basys 3 XADC Demo.” Digilent. Accessed on June 3, 2023.

<https://digilent.com/reference/programmable-logic/basys-3/demos/xadc>

“Binary to BCD Converter.” All About FPGA. Accessed on June 3, 2023. <https://allaboutfpga.com/vhdl-code-for-binary-to-bcd-converter/>

artvnb. “Basys-3-Master.xdc.” Github. Accessed on June 3, 2023. <https://github.com/Digilent/digilent-xdc/blob/master/Basys-3-Master.xdc>